4. Структура данных (The data structure)

Из статьи: Mikael Degermark, Andrej Brodnik, Svante Carlsson, Stephen Pink. «Small Forwarding Tables for Fast Routing Lookups»

Таблица пересылки по существу представляет собой дерево с тремя уровнями. Поиск на одном уровне требует от одного до четырёх обращений к памяти. Следовательно, максимальное количество обращений к памяти равно двенадцати. Однако на проверенных нами таблицах маршрутизации подавляющее большинство операций поиска требует обращения только к одному или двум уровням, поэтому наиболее вероятное количество обращений к памяти составляет восемь или менее.

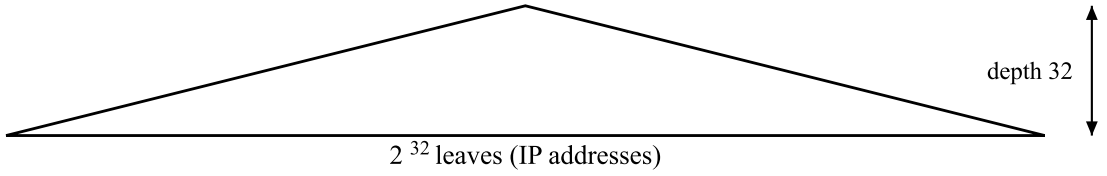


Figure 3: Binary tree spanning the entire IP address space.

Для понимания структуры данных представьте двоичное дерево, охватывающее всё пространство IP-адресов (рис. 3). Его высота равна 32, а количество листьев составляет 2^{32} — по одному на каждый возможный IP-адрес. Префикс записи таблицы маршрутизации задаёт путь в дереве, заканчивающийся в некотором узле. Все IP-адреса (листья) в поддереве с корнем в этом узле должны маршрутизироваться в соответствии с этой записью маршрутизации. Таким образом, каждая запись таблицы маршрутизации определяет диапазон IP-адресов с идентичной информацией о маршрутизации.

Если один и тот же IP-адрес покрывается несколькими записями маршрутизации, применяется правило наибольшего совпадения (rule of the longest match); оно гласит, что для заданного IP-адреса должна использоваться запись маршрутизации с наиболее длинным совпадающим префиксом. Эта ситуация проиллюстрирована на рис. 4: запись маршрутизации e1 скрыта записью e2 для адресов в диапазоне r.

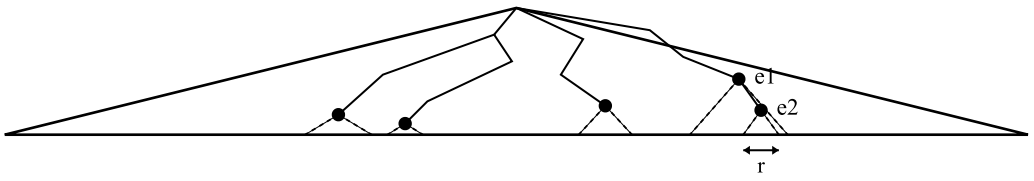


Figure 4: Routing entries defining ranges of IP addresses.

Таблица пересылки представляет собой представление двоичного дерева, образованного всеми записями маршрутизации. Оно называется *деревом префиксов (prefix tree)*. Мы требуем, чтобы дерево префиксов было полным, то есть чтобы каждый узел дерева имел либо двух потомков, либо ни одного. Узлы с одним потомком должны быть расширены так, чтобы иметь двух потомков; добавляемые таким образом потомки всегда являются листьями, а их информация о следующем узле (next-hop) совпадает с next-hop ближайшего предка, обладающего информацией о next-hop, либо принимает значение «не определено» («undefined»), если такой предок отсутствует.

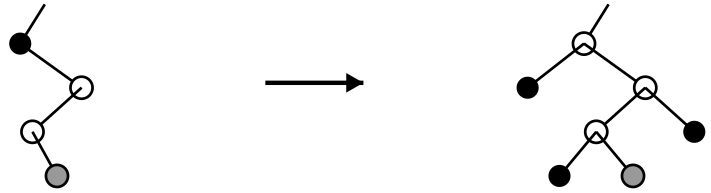


Figure 5: Expanding the prefix tree to be complete.

Эта процедура, проиллюстрированная на рис. 5, увеличивает количество узлов в дереве префиксов, но позволяет построить компактную таблицу пересылки. Заметьте, что для построения таблицы пересылки на самом деле нет необходимости физически строить дерево префиксов. Мы используем дерево префиксов для упрощения объяснения. Таблица пересылки может быть построена за один проход по всем записям маршрутизации.

Набор записей маршрутизации разбивает пространство IP-адресов на множества IP-адресов. Задача поиска надлежащей информации о маршрутизации схожа с более общей задачей принадлежности интервальному множеству [12]. Однако в нашем случае интервалы определяются узлами в полном дереве префиксов и, следовательно, обладают свойствами, которые мы можем использовать для получения ещё более компактной структуры данных. Например, каждый диапазон IP-адресов имеет длину, равную степени двойки.

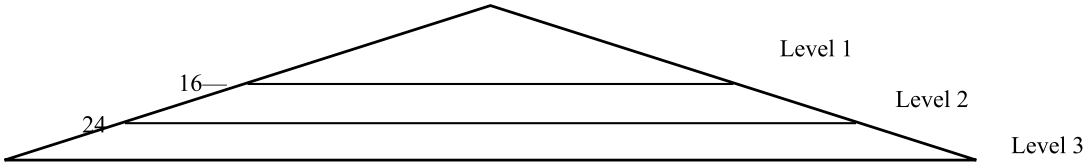


Figure 6: The three levels of the data structure.

Как показано на рис. 6, уровень один структуры данных охватывает дерево префиксов до глубины 16, уровень два охватывает глубины от 17 до 24, а уровень три — глубины от 25 до 32. Всюду, где часть дерева префиксов опускается ниже уровня 16, чанк (блок) второго уровня описывает эту часть дерева. Аналогично, чанки на третьем уровне описывают части дерева префиксов, расположенные глубже 24. Результатом поиска на одном уровне структуры данных является либо индекс в таблице следующих узлов (next-hop table), либо индекс в массиве чанков для следующего уровня.

4.1. Основной результат (Core result)

Наш основной результат заключается в том, что мы можем представить полное двоичное дерево высоты h, используя всего один бит на каждый возможный лист на глубине h, плюс один базовый индекс на каждые 64 возможных листа, плюс информацию, хранящуюся в листьях. Для $h > 6$ размер в байтах дерева с l листьями, содержащими информацию размера d, равен:

$$2^{h-3} + b \times 2^{h-6} + l \times d \quad (1)$$

где b — размер базового индекса. При двухбайтовых базовых индексах дерево высоты 8 (чанк) требует 40 байт плюс информация листьев, а дерево высоты 16 требует 10 Кбайт плюс информация листьев.

Для достижения таких малых размеров требуется дополнительная таблица размером 5408 байт. Таблицу можно сделать меньше — 1352 байта, но тогда типовым процессорам потребуется больше инструкций при её использовании.

4.2. Уровень 1 структуры данных (Level 1 of the data structure)

Первый уровень по существу представляет собой узел дерева с 64K потомками. Он охватывает дерево префиксов до глубины 16.

Представьте срез через дерево префиксов на глубине 16. Срез представляется битовым вектором, содержащим по одному биту на каждый возможный узел на глубине 16. Для этого требуется 2^{16} бит = 64K бит = 8 Кбайт. Чтобы найти бит, соответствующий начальной части IP-адреса, старшие 16 бит адреса используются в качестве индекса в битовом векторе.

Заголовки (Heads). Когда в дереве префиксов имеется узел на глубине 16, соответствующий бит в векторе устанавливается в 1. Также, когда дерево содержит лист на глубине менее 16, устанавливается младший бит в интервале, покрываемом этим листом. Все остальные биты равны нулю. Бит в битовом векторе, таким образом, может быть:

- **единицей**, означающей, что дерево префиксов продолжается ниже среза; *корневой заголовок (root head)* (биты 6, 12 и 13 на рис. 7), или
- **единицей**, представляющей лист на глубине 16 или менее; *подлинный заголовок (genuine head)* (биты 0, 4, 7, 8, 14 и 15 на рис. 7), или
- **нулём**, что означает, что данное значение является *элементом (member)* диапазона, покрываемого листом на глубине менее 16 (биты 1, 2, 3, 5, 9, 10 и 11 на рис. 7). Элементы имеют тот же next-hop, что и наибольший заголовок, меньший данного элемента.

Битовый вектор делится на битовые маски длиной 16. Таких масок насчитывается $2^{12} = 4096$.

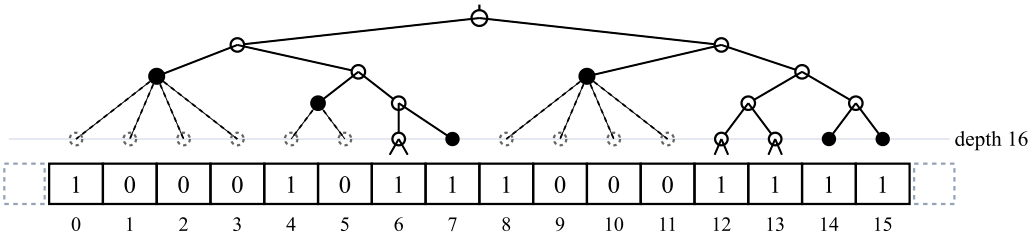


Figure 7: Part of cut with corresponding bit-vector

Информация заголовков (Head information). Для подлинных заголовков нам необходимо хранить индекс в таблице следующих узлов (next-hop table). Элементы будут использовать тот же next-hop, что и наибольший заголовок, меньший данного элемента. Для корневых заголовков нам необходимо сохранять индекс чанка второго уровня, представляющего соответствующее поддерево.

Информация заголовков кодируется в 16-битных указателях, хранящихся последовательно в массиве. Два бита каждого указателя кодируют тип указателя, а оставшиеся 14 бит либо образуют индекс в таблице next-hop, либо индекс в массиве, содержащем чанки второго уровня. Обратите внимание, что с битовой маской связано столько указателей, сколько в ней установлено единичных бит.

Поиск групп указателей (Finding pointer groups). Рис. 8 иллюстрирует, как структура данных для поиска указателей соотносится с битовыми масками. Структура данных состоит из массива кодовых слов — столько же, сколько битовых масок, — плюс массив базовых индексов, по одному на каждые четыре кодовых слова. Кодовые слова состоят из 10-битного значения (r1, r2, ...) и 6-битного смещения (0, 3, 10, 11, ...).

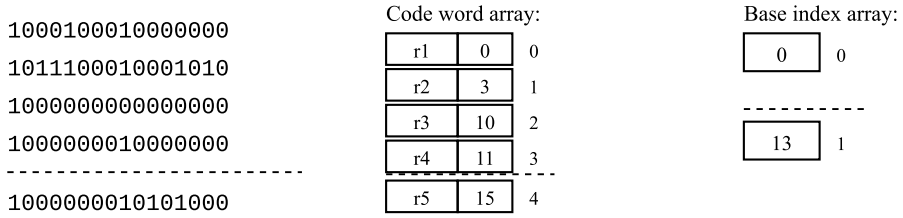


Figure 8: Bit-masks vs code words and base indices.

Первая битовая маска на рис. 8 имеет три установленных бита. Второе кодовое слово, таким образом, имеет смещение три, поскольку необходимо пропустить три указателя, чтобы найти первый указатель, связанный с этой битовой маской. Вторая битовая маска имеет 7 установленных бит, и, следовательно, смещение в третьем кодовом слове равно 3 + 7 = 10.

После четырёх кодовых слов значение смещения может стать слишком большим, чтобы его можно было представить 6 битами. Поэтому для нахождения группы указателей совместно со смещением используется базовый индекс. На уровне 1 структуры данных может быть не более 64K указателей, поэтому базовые индексы должны иметь размер не более 16 бит (2^{16} = 64K). На рис. 8 второй базовый индекс равен 13, поскольку в первых четырёх битовых масках содержится 13 установленных бит.

Это объясняет, как находится группа указателей. Первые 12 бит IP-адреса служат индексом соответствующего кодового слова, а первые 10 бит — индексом в массиве базовых индексов.

Таблица отображения (Maptable). Остаётся объяснить, как найти нужный указатель внутри группы указателей. Для этого и предназначено 10-битное значение (r1, r2, ... на рис. 8). Это значение является индексом в таблице, которая сопоставляет номера бит в IP-адресе со смещениями указателей. Поскольку длина битовых масок составляет 16 бит, можно было бы подумать, что таблице требуется 64K записей. Однако битовые маски генерируются из полного дерева префиксов, поэтому возможны далеко не все комбинации 16 бит.

Ненулевая битовая маска длины $2n$ может представлять собой любую комбинацию двух битовых масок длины n либо битовую маску со значением 1. Пусть $a(n)$ — количество возможных ненулевых битовых масок длины 2^n. $a(n)$ определяется рекуррентным соотношением:

$$a(0) = 1, \qquad a(n) = 1 + a(n-1)^2 \tag{2}$$

Таким образом, количество возможных битовых масок длины 16 составляет $a(4) + 1 = 678$; дополнительная единица обусловлена тем, что битовая маска может быть нулевой. Индексу в таблице с записью для каждой битовой маски, следовательно, требуется всего 10 бит.

Мы храним такую таблицу, *maptable*, для отображения номеров бит внутри битовой маски в 4-битные смещения. Смещение указывает, сколько указателей нужно пропустить, чтобы найти искомый, то есть оно равно количеству установленных бит с индексами, меньшими данного битового индекса. Эти смещения одинаковы для всех таблиц пересылки, независимо от того, какие значения имеют сами указатели. Таблица *maptable* константна, она генерируется раз и навсегда.

Поиск (Searching). Шаги на рис. 10 необходимы для поиска на первом уровне структуры данных; массив кодовых слов называется `code`, а массив базовых индексов — `base`.

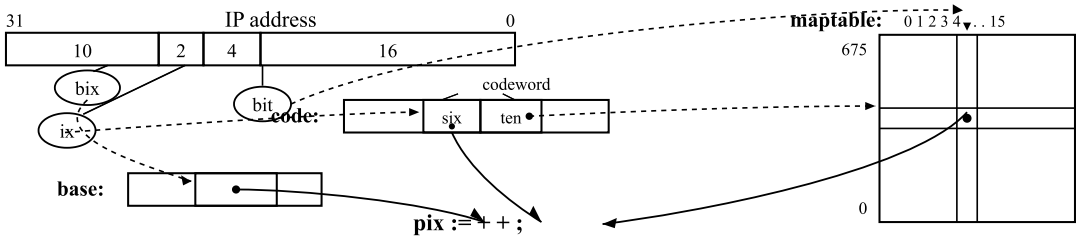


Figure 9: Finding the pointer index.

```
ix       := high 12 bits of IP address
bix      := high 10 bits of IP address
bit      := low 4 of high 16 bits of IP address
codeword := code [ix]
ten      := ten bits from codeword
six      := six bits from codeword
pix      := base [bix] + six + maptable [ten] [bit]
pointer  := level1_pointers [pix]
```

Figure 10: Steps to search the first level

Рис. 9 иллюстрирует эту процедуру. Индекс кодового слова, `ix`, индекс базового индекса, `bix`, и номер бита, `bit`, сначала извлекаются из IP-адреса. Затем извлекается кодовое слово, и две его части разделяются на `ten` и `six`. Индекс указателя, `pix`, затем получается путём сложения базового индекса, 6-битного смещения `six` и смещения указателя, полученного выборкой столбца `bit` из строки `ten` таблицы `maptable`. После того как указатель извлечён из массива указателей, он проверяется, чтобы определить, найден ли next-hop или поиск следует продолжить на следующем уровне.

Код чрезвычайно прост. Несколько побитовых извлечений, обращений к массивам и сложений — это всё, что требуется. Не требуется никаких инструкций умножения или деления, за исключением неявных умножений при индексации массивов.

Всего для поиска на первом уровне необходимо обратиться к 7 байтам: двухбайтовое кодовое слово, двухбайтовый базовый адрес, один байт (на самом деле 4 бита) в maptable и, наконец, двухбайтовый указатель. Размер первого уровня составляет 8 Кбайт для массива кодовых слов, 2 Кбайта для массива базовых индексов, плюс некоторое количество указателей. Требуемые 5,3 Кбайта для maptable разделяются между всеми тремя уровнями.

4.2.1. Оптимизации на уровне 1 (Optimizations at level 1)

Когда битовая маска равна нулю или в ней установлен единственный бит, указатель обязательно является индексом в таблице next-hop. Такие указатели могут кодироваться непосредственно в кодовом слове, и поэтому таблице maptable не требуются записи для битовых масок единицы и нуля. Количество записей в maptable тем самым сокращается до 676 (индексы от 0 до 675). Когда десять бит в кодовом слове (`ten` выше) больше 675, кодовое слово представляет прямой индекс в таблице next-hop. Шесть бит из кодового слова используются как младшие 6 бит индекса, а (*ten* − *676*) являются старшими битами индекса. Такое кодирование позволяет адресовать максимум $(1024 - 676) \times 2^6 = 22272$ индексов next-hop, что превышает 16K, на которые мы рассчитываем. Эта оптимизация устраняет три обращения к памяти, когда запись маршрутизации располагается на глубине 12 или выше, и значительно сокращает число указателей в массиве указателей. Платой за это являются операция сравнения и условный переход.

4.3. Уровни 2 и 3 структуры данных (Levels 2 and 3 of the data structure)

Уровни два и три структуры данных состоят из чанков. Чанк охватывает поддерево высоты 8 и может содержать не более $2^8 = 256$ заголовков. Корневой заголовок на уровне $n - 1$ указывает на чанк на уровне n.

Существует три разновидности чанков в зависимости от того, сколько заголовков содержит воображаемый битовый вектор. Когда заголовков:

- **1–8 заголовков:** чанк является *разреженным (sparse)* и представляется массивом 8-битных индексов заголовков плюс восемь 16-битных указателей — всего 24 байта.
- **9–64 заголовков:** чанк является *плотным (dense)*. Он представляется аналогично уровню 1, за исключением числа базовых индексов. Разница в том, что для всех 16 кодовых слов необходим только один базовый индекс, поскольку 6-битные смещения могут охватить все 64 указателя. Всего требуется 34 байта плюс от 18 до 128 байт для указателей.
- **65–256 заголовков:** чанк является *очень плотным (very dense)*. Он представляется аналогично уровню 1: 16 кодовых слов и 4 базовых индекса дают в сумме 40 байт. Кроме того, от 65 до 256 указателей требуют от 130 до 512 байт.

Плотные и очень плотные чанки просматриваются аналогично первому уровню. Для разреженных чанков от 1 до 8 значений располагаются в порядке убывания. Чтобы избежать неблагоприятного худшего случая при поиске, проверяется четвёртое значение, чтобы определить, находится ли искомый элемент среди первых четырёх или последних четырёх элементов. После этого линейное сканирование определяет индекс искомого элемента, и указатель с этим индексом может быть извлечён. Первый элемент, меньший или равный ключу поиска, является искомым элементом. Для поиска в разреженном чанке требуется доступ максимум к 7 байтам.

4.3.1. Оптимизации на уровнях два и три (Optimizations at levels two and three)

Плотные и очень плотные чанки оптимизируются аналогично уровню 1, как описано в подразделе 4.2.1. В разреженных чанках последовательные заголовки могут объединяться и представляться наименьшим из них, если их next-hop идентичны. При принятии решения о том, является ли чанк разреженным или плотным, это объединение принимается во внимание, так что чанк считается разреженным, если число объединённых заголовков составляет 8 или менее. Многие из листьев, добавленных для того, чтобы сделать дерево полным, будут следовать по порядку и иметь одинаковые next-hop. Заголовки, соответствующие таким листьям, будут объединены в разреженных чанках.

Эта оптимизация сдвигает распределение чанков от более крупных плотных чанков в сторону более компактных разреженных чанков. Для больших таблиц размер таблицы пересылки обычно уменьшается на 5–15 процентов.

4.4. Ограничения роста в текущей конструкции (Growth limitations in the current design)

Структура данных может обеспечить значительный рост числа записей маршрутизации. В текущей конструкции имеются три ограничения.

1. Количество чанков каждого вида ограничено величиной $2^{14} = 16384$ на уровень.
Таблица 1 показывает, что это примерно в 16 раз больше, чем используется в настоящее время. Если данный предел когда-либо будет превышен, структура данных может быть модифицирована так, чтобы указатели кодировались иначе для выделения большего пространства под индексы, либо чтобы размер указателя был увеличен.

2. Число указателей на уровнях два и три ограничено размером базовых индексов.
Текущая реализация использует 16-битные базовые индексы и способна выдержать рост в 3–5 раз. Если предел будет превышен, не составит труда увеличить размер базовых указателей до трёх байт. Размер чанка в этом случае увеличится на 3 процента для плотных чанков и на 10 процентов для очень плотных чанков. На разреженные чанки это не влияет.

3. Число различных next-hop ограничено величиной $2^{14} = 16384$.
Если этот предел будет превышен, не все индексы next-hop смогут кодироваться непосредственно в кодовых словах, как объяснялось в подразделе 4.2.1. Возможно избежать сохранения указателя, когда битовая маска равна нулю. Однако, когда битовая маска имеет один заголовок, сохранение указателя необходимо. Следовательно, размер структуры данных увеличится, поскольку потребуется один указатель на интервал, а указатели станут больше.

В заключение: при небольших модификациях структура данных способна выдержать многократное увеличение числа записей маршрутизации.